

COS
Disk Operating System

User's Guide and Technical Reference

Version 1.00
for the pc86 Personal Computer

CUCSICORP

PERSONAL COMPUTER SOFTWARE

First Edition (October 1981)

Changes are made periodically to the information herein; any such changes will be reported in subsequent revisions.

This manual describes COS Version 1.00, the disk operating system for the pc86 personal computer, its command processor, COMMAND.COM, its text editor, EDIT, and its assembler, ASM. The BASIC language supplied with COS, CBasic, has a manual of its own.

The programs described in this manual are supplied on the COS System Diskette. Read the whole of Chapter 2, "Getting Started," before you use COS for the first time.

Products are not stocked at the address below. Comments on this manual may be addressed to CucsiCorp, Personal Computer Software, Publications Department. CucsiCorp may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligations whatever.

© Copyright CucsiCorp 1981

COS and CucsiCorp are trademarks of CucsiCorp.

Preface

This manual tells you how to use COS, the CucsiCorp disk operating system, and how to write programs for it. It is written for two kinds of readers:

- If you want to **use** COS — start it, look after your diskettes, copy and erase files, run programs — read Part One, Chapters 1 to 5. You do not need to know anything about programming.
- If you want to **write programs** for COS in 8086 assembly language, read Part One first, then Part Three, Chapters 9 and 10, which show how to write programs with the editor and the assembler on the computer itself, and Part Two, Chapters 6 to 8, and the appendixes.

Organization of This Manual

Chapter 1	Introduction: what COS is and what you need to use it.
Chapter 2	Getting Started: starting COS, the prompt, the keyboard, date and time.
Chapter 3	Files and Diskettes: drives, file names, global file name characters, looking after diskettes.
Chapter 4	COS Commands: every command, in alphabetic order, with examples.
Chapter 5	Messages: what each message means and what to do about it.
Chapter 6	Programming COS: writing, assembling and running .COM and .EXE programs; the program segment prefix, memory, files and errors.
Chapter 7	Interrupt and Function Reference: INT 20h to 2Fh and every INT 21h function.
Chapter 8	Technical Information: memory map, diskette format, directory entries, disk buffers, compatibility.
Chapter 9	EDIT, the Text Editor: writing and changing programs and other text files.
Chapter 10	ASM, the Assembler: turning a program into a .COM file, and the assembly language it accepts.
Appendix A	Error Codes.
Appendix B	The Keyboard: keys, Ctrl combinations and extended codes.

Appendix C	The Character Set: code page 437.
Appendix D	BIOS Services and the Hardware.

Notation

In this manual, the following conventions are used in command formats and examples:

CAPITALS	Words in capital letters must be typed as shown, although you may type them in small letters: COS does not mind.
italics	Words in italics are items you supply, for example a file name.
[ ]	Square brackets mean the item inside is optional. Do not type the brackets.
d:	A drive letter followed by a colon: A: or B:.
Enter	A key you press. **Ctrl**-**C** means: hold down Ctrl and press C.
dir	In the screen examples, what you type is printed in bold and underlined; everything else is what COS displays.
3Dh	Numbers that end in "h" are hexadecimal (base 16). Others are decimal.

The screen examples in this manual were made on the machine itself. Your display will show the same text, although the sizes and free space shown for your diskettes will differ.

Contents

PART ONE: USING COS

Chapter 1. Introduction **1-1**
- What COS Is 1-2
- What You Need 1-2
- What COS Can Do 1-2
- What Is on the System Diskette 1-3
- How to Use This Manual 1-3

Chapter 2. Getting Started **2-1**
- Starting COS 2-2
- The Prompt 2-2
- Typing Commands 2-3
- Looking at a Diskette 2-3
- Running a Program 2-4
- Entering the Date and Time 2-4
- Stopping a Program 2-5
- Turning Off the Computer 2-5

Chapter 3. Files and Diskettes **3-1**
- Files 3-2
- File Names 3-2
- Global Filename Characters 3-3
- Directories 3-4
- Diskettes 3-5

Chapter 4. COS Commands **4-1**
- About the Commands 4-2
- CD (Change Directory) 4-2
- CLS (Clear Screen) 4-3
- COPY 4-3
- DATE 4-4
- DEL (Delete) 4-5
- DIR (Directory) 4-6
- ECHO 4-7
- EXIT 4-7
- MD (Make Directory) 4-7
- RD (Remove Directory) 4-8
- PAUSE 4-9

REM (Remark) 4-9
REN (Rename) 4-9
SET 4-10
TIME 4-11
TYPE 4-11
VER (Version) 4-12
VOL (Volume) 4-12
d: (Change the Default Drive) 4-12
Running Programs 4-12

Chapter 5. Messages 5-1
Batch Files 5-2
About the Messages 5-2
Disk Errors 5-2
Messages in Alphabetic Order 5-3

PART TWO: PROGRAMMING

Chapter 6. Programming COS 6-1
Introduction 6-2
Tools 6-2
Your First Program 6-2
.COM Programs 6-3
.EXE Programs 6-4
The Program Segment Prefix 6-5
The Environment 6-5
Memory 6-6
Files and Handles 6-6
Errors 6-7
Ctrl-C 6-7
Running Another Program 6-8
Terminate and Stay Resident 6-8
The Processor 6-9
Rules for Well-Behaved Programs 6-9

Chapter 7. Interrupt and Function Reference 7-1
The COS Interrupts 7-2
Calling Conventions 7-2
Functions in Numeric Order 7-2
Console Functions 7-4
Disk and Directory Functions 7-6

File Functions 7-8
Program Functions 7-12
Memory Functions 7-13
Other Functions 7-14

Chapter 8. Technical Information 8-1
Memory Map 8-2
How COS Starts 8-2
Diskette Format 8-3
The File Allocation Table 8-4
Directory Entries 8-4
Disk Buffering 8-5
Critical Errors 8-5
System Limits 8-5
Compatibility 8-6

PART THREE: PROGRAM DEVELOPMENT

Chapter 9. EDIT, the Text Editor 9-1
Introduction 9-2
EDIT 9-2
The Screen 9-2
Moving the Cursor 9-3
Typing and Changing Text 9-4
Moving Lines 9-4
Finding Text 9-4
Saving and Leaving 9-5
Text Files 9-5
EDIT Messages 9-5

Chapter 10. ASM, the Assembler 10-1
Introduction 10-2
Writing a Program 10-2
ASM 10-3
How ASM Works 10-4
Assembling COS 10-4
The Source File 10-5
ASM Messages 10-8

APPENDIXES

Appendix A. Error Codes **A-1**
Function Error Codes A-2
Return Codes A-2

Appendix B. The Keyboard **B-1**
Keys at the Prompt B-2
Control Characters B-2
Extended Codes B-2

Appendix C. The Character Set **C-1**
Code Page 437 ..... C-2

Appendix D. BIOS Services and the Hardware **D-1**
BIOS Services D-2
Hardware D-2

Quick Reference

1

Introduction

CONTENTS

What COS Is 1-2
What You Need 1-2
What COS Can Do 1-2
What Is on the System Diskette 1-3
How to Use This Manual 1-3

1

What COS Is

COS, the CucsiCorp Operating System, is the disk operating system for the pc86 personal computer. It is the program that looks after the computer for you: it keeps your programs and information in *files* on diskettes, starts your programs, gives them the memory and the files they ask for, and lets you copy, rename, display and erase files with simple commands.

COS has two parts:

COS.SYS	The operating system itself. It is read from the diskette when you start the computer and stays in memory all the time. Programs ask it for services through interrupt 21h.
COMMAND.COM	The command processor. It shows the prompt, reads what you type and carries out your commands. It is an ordinary program: if it ever ends, COS starts it again.

A small third part, the *boot record*, lives in the first sector of the System Diskette. It is what the computer reads first; its only job is to find COS.SYS on the diskette and start it.

What You Need

- A pc86 personal computer, with the 8086 microprocessor and 640 KB of memory.
- A video card and a monitor. COS uses the 80-column text mode.
- A keyboard.
- A diskette adapter with one or two 3½-inch diskette drives. The first drive is drive A:, the second drive B:.
- The COS System Diskette, and blank formatted diskettes for your own work. COS reads and writes 1.44 MB (high density) and 720 KB (double density) diskettes.

The computer must boot from drive A:. The System Diskette goes in drive A:; drive B:, if you have one, holds a second diskette, usually the one with your work.

What COS Can Do

- Run programs: .COM and .EXE files, one at a time, with up to 583 KB of memory for each program.

- Keep files on diskettes, in directories: up to 224 files in the main directory of a 1.44 MB diskette (112 on a 720 KB diskette), and as many as fit in the directories you make inside it, each file up to the size of the diskette.
- Copy, rename, erase and display files, and list what is on a diskette.
- Keep the date and the time of day, and record them on every file you write.
- Notice when you change the diskette in a drive, and tell you when a diskette cannot be read or written.

COS provides the services of the version 3.30 disk operating system that programs most often use, with the same function numbers, so that programs written for those services run without change. Chapter 8 lists what COS leaves out.

What Is on the System Diskette

COS.SYS	The operating system.
COMMAND.COM	The command processor.
EDIT.COM	The text editor (Chapter 9).
ASM.COM	The assembler (Chapter 10).
HELLO.ASM	A short program to try the editor and the assembler with.
Programs	Your System Diskette may also hold sample programs, for example HELLO.COM, NUMBERS.COM, FACTOR.COM and SORT.COM. Type their names at the prompt to run them.

WARNING: Never erase or rename COS.SYS or COMMAND.COM on your System Diskette. Without them the computer cannot start COS.

How to Use This Manual

If you are new to COS, read Chapters 2 and 3 at the computer, trying each example as you read. Chapter 4 describes every command; you do not need to learn them all at once. Keep Chapter 5 at hand: when COS shows a message you do not understand, look it up there. The Quick Reference at the end of the manual summarizes the commands and keys on one page.

This page intentionally left blank.

2

Getting Started

CONTENTS

Starting COS 2-2
The Prompt 2-2
Typing Commands 2-3
Looking at a Diskette 2-3
Running a Program 2-4
Entering the Date and Time 2-4
Stopping a Program 2-5
Turning Off the Computer 2-5

Starting COS

1. Insert the COS System Diskette in drive A:, label side up.
2. Turn on the monitor, then the computer. If the computer is already on, press its Reset button.

The computer tests itself, reads the boot record from the diskette, and the boot record loads COS. After a moment you see:

```
CucsiCorp COS Version 1.00
(C) Copyright CucsiCorp 1981
A:\>
```

COS is now ready for your commands. If the diskette has a batch file named AUTOEXEC.BAT in its root directory, COS runs it first (see "Batch Files" in Chapter 4), so a diskette can start a program by itself.

If instead you see one of the following, the computer could not start COS:

No boot device available.	There is no diskette in drive A:, or no diskette adapter. Insert the System Diskette and press Reset.
The diskette in drive A: is not bootable.	The diskette in drive A: has no boot record. It is not a System Diskette.
Non-system disk	The diskette is formatted but has no operating system. Replace it with the System Diskette and press any key.
COS.SYS is missing	The diskette has the COS boot record but not COS.SYS. Use another System Diskette.
Disk error	The diskette could not be read. Try again; if it fails again, the diskette is damaged.

The Prompt

The prompt, **A:\>**, means that COS is waiting for a command. The letter tells you the *default drive*: the drive COS uses when you do not name one. To make drive B: the default drive, type **b:** and press **Enter**:

```
A:\>b:

B:\>a:

A:\>
```


Typing Commands

Type a command after the prompt and press **Enter**. Nothing happens until you press Enter, so you can correct mistakes first:

Backspace	Erases the last character you typed.
Esc	Cancels the whole line. COS displays a backslash (\), moves to the next line, and you can type the line again.
Enter	Gives the line to COS.
Ctrl-**C**	Cancels the line, or stops the command or program that is waiting for you to type something. COS displays ^C and a new prompt.

You may type commands in capital or small letters. A line holds up to 127 characters; if you type more, the computer beeps and ignores the extra keys.

Try the VER command, which displays the version of COS:

```
A:\>ver

CucsiCorp COS Version 1.00

A:\>
```

If COS does not recognize what you typed, it says so:

```
A:\>helo
Bad command or file name
```

Looking at a Diskette

The DIR command lists the files on a diskette:

```
A:\>dir

 Volume in drive A is COS
 Directory of  A:\
```

```
COS      SYS    12902   1-01-80  12:00a
COMMAND  COM     7035   1-01-80  12:00a
ASM      COM    17945   1-01-80  12:00a
EDIT     COM     5410   1-01-80  12:00a
HELLO    ASM      299   1-01-80  12:00a
HELLO    COM       31   1-01-80  12:00a
NUMBERS  COM       69   1-01-80  12:00a
FACTOR   COM       81   1-01-80  12:00a
SORT     COM      124   1-01-80  12:00a
README   TXT       94   1-01-80  12:00a
LETTER   TXT       47   1-01-80  12:00a
       11 File(s)   1409536 bytes free
```

Each line shows a file's name, its extension, its size in bytes, and the date and time it was last written. The last line tells you how much room is left on the diskette.

Running a Program

To run a program, type its name, without the extension:

```
A:\>hello
Hello from the 8086!

A:\>factor
1
1
2
6
24
120
720
5040
40320
```

When the program ends, the prompt comes back. COS looks for the program on the default drive, or on the drive you name before it: **b:game** runs GAME.COM (or GAME.EXE) from drive B:.

Entering the Date and Time

The computer has no battery clock. Each time you start COS, the date is set to Tuesday, 1 January 1980, and the time to midnight. COS records the date and time on every file you write, so set them first with the DATE and TIME commands:

```
A:\>date
Current date is Tue  1-01-1980
Enter new date (mm-dd-yy): 10-14-81

A:\>time
Current time is  0:00:00.32
Enter new time: 9:30
```


The time is kept on the 24-hour clock: type 13:15 for a quarter past one in the afternoon. To keep the date or time shown, just press **Enter**.

Stopping a Program

Press **Ctrl**-**C** to stop a program while it is waiting for you to type something. COS ends the program and shows the prompt. A program that never reads the keyboard cannot be stopped this way; press the computer's Reset button to start again.

Turning Off the Computer

Before you turn off the computer or remove a diskette, wait until the prompt is displayed and the drive light is out. COS writes some information to the diskette only when a program finishes with a file; removing the diskette before that may leave the file incomplete.

This page intentionally left blank.

3

Files and Diskettes

CONTENTS

Files 3-2
File Names 3-2
Global Filename Characters 3-3
Directories 3-4
Diskettes 3-5

Files

Everything COS keeps on a diskette is kept in *files*. A program is a file; a letter, a list or a game's high scores is a file. Every file has a name, and every diskette has a *directory*, a list of the files on it with their names, sizes and dates.

File Names

A file name has two parts: the *filename*, one to eight characters, and an optional *extension* of up to three characters, separated by a period:

```
LETTER.TXT      filename LETTER, extension TXT
COMMAND.COM     filename COMMAND, extension COM
SCORES          no extension
```

The characters you can use are the letters A to Z, the digits 0 to 9, and these special characters:

```
$ & # @ ! % ' ( ) - { } ~ ` ^ _
```

Small letters are changed to capitals, so **letter.txt** and **LETTER.TXT** are the same file. Extra characters are ignored: **CORRESPONDENCE.TEXT** is the file CORRESPO.TEX.

Extensions

The extension usually says what kind of file it is. COS gives these extensions a meaning:

.COM	A program, in the simple .COM form (up to 64 KB).
.EXE	A program, in the .EXE form, which may be larger and use several segments.
.SYS	Part of the operating system: COS.SYS.
.BAT	A batch file: commands that COS runs one after another (Chapter 4).

Other common extensions are .TXT for text, .ASM for assembly language programs and .DAT for data, but you may use any extension you like.

File Specifications

To name a file on a drive other than the default drive, put the drive letter and a colon in front of it. This is a *file specification*:

```
B:LETTER.TXT
```

A file specification can also say which *directory* the file is in, B:\LETTERS\JUNE.TXT. Directories are explained below.

Reserved Names

Some names stand for devices, not files, and cannot be used as filenames, with any extension:

CON	The console: the keyboard for input, the display for output.
NUL	A dummy device: what is written to it disappears, and reading from it gives an immediate end of file.
AUX, PRN, COM1, COM2, LPT1, LPT2, CLOCK$	Serial, printer and clock devices. The pc86 computer does not have them; opening them is refused ("Access denied").

For example, **copy readme.txt con** would display the file, as TYPE does.

Global Filename Characters

Two characters, ? and *, let one file specification stand for several files. They are called *global filename characters*, or *wildcards*.

The ? Character

A ? in a filename or extension matches any character in that position:

```
DIR REPORT?.TXT     lists REPORT1.TXT, REPORT2.TXT, REPORTA.TXT ...
```

The * Character

An * matches any characters from its position to the end of the filename or the extension:

```
DIR *.TXT           every file with the extension TXT
DIR R*.*            every file whose name begins with R
COPY *.* B:         every file, to drive B:
```

DIR, COPY, DEL and REN accept global characters. Other commands, and programs you run, need a single file name.

Directories

A diskette with many files is easier to use when they are kept in groups. COS lets you make *directories* for them, the way a filing cabinet has folders: one for your letters, one for your games, one for a program you are writing.

Every diskette starts with one directory, the *root directory*. You can make directories in it, and directories in those, as many as you like. A directory has a name like a file (eight characters, and an extension if you wish), and appears in the listing of the directory that holds it as **<DIR>**:

```
A:\              the root directory
   COMMAND.COM
   HELLO.COM
   GAMES         a directory
      SPACE.COM
      SCORES.DAT
   LETTERS       another one
      JUNE.TXT
```

The Current Directory

COS remembers, for each drive, a *current directory*: the one where files are looked for and made when you give no directory. When you start COS, it is the root directory. The prompt shows the current directory of the default drive, and the CD command changes it:

```
A:\>md letters

A:\>copy letter.txt letters
        1 File(s) copied

A:\>cd letters

A:\LETTERS>dir

 Volume in drive A is COS
 Directory of  A:\LETTERS

.              <DIR>     1-01-80  12:00a
..             <DIR>     1-01-80  12:00a
LETTER   TXT       47   1-01-80  12:00a
        3 File(s)   1409024 bytes free

A:\LETTERS>
```

Every directory but the root holds two entries COS makes by itself: `.`, the directory itself, and `..`, the directory that holds it.

Paths

A *path* names a directory by the directories that lead to it, separated by backslashes (\). A path that starts with a backslash starts at the root directory; any other starts at the current directory. `..` goes up one level:

\GAMES\SPACE.COM	SPACE.COM in the directory GAMES of the root directory.
GAMES\SPACE.COM	The same, if the root directory is the current directory.
..\NOTES.TXT	NOTES.TXT in the directory above the current one.
B:\LETTERS	The directory LETTERS on the diskette in drive B:.
B:JUNE.TXT	JUNE.TXT in the current directory of drive B:.

Every command that takes a file name takes a path too: **type \letters\june.txt**, **copy *.com games**, **del b:\old*.***. A path may be up to 63 characters long.

When the diskette in a drive is changed, the current directory of that drive goes back to the root directory.

Diskettes

	High density	Double density
Capacity	1,474,560 bytes (1.44 MB)	737,280 bytes (720 KB)
Space for files	1,457,664 bytes	730,112 bytes
Files in the directory	224	112
Allocation unit	512 bytes	1,024 bytes

Space on a diskette is given to files in *allocation units*, so a file of 10 bytes uses 512 bytes on a 1.44 MB diskette. The free space DIR shows is a multiple of the allocation unit.

Preparing Diskettes

COS works with diskettes that have been *formatted*: prepared with tracks, sectors, an empty directory and a file allocation table. COS Version 1.00 does not format diskettes itself: use the formatted blank diskettes supplied for the pc86, or a diskette COS has already used, after erasing its files with DEL *.*. To make a new System Diskette, copy COS.SYS, COMMAND.COM and the programs you want onto a diskette that already has the COS boot record, such as a copy of your System Diskette.

Write Protection

A diskette has a write-protect tab. When the tab is open, COS can read the diskette but cannot change it, so files on it cannot be erased by mistake. If you try to write to a protected diskette, COS says:

```
A:\>copy letter.txt b:

Write protect error writing drive B
Abort, Retry, Fail?F
File creation error
        0 File(s) copied
```

Close the tab and type **R** to retry, or type **F** to give up. Chapter 5 explains the three answers.

Changing Diskettes

You may change the diskette in a drive whenever the prompt is displayed. COS notices the change the next time it uses the drive, and reads the new diskette's directory:

```
A:\>dir b:

 Volume in drive B is WORK
 Directory of  B:\

File not found

A:\>dir b:

 Volume in drive B is COS
 Directory of  B:\

COS      SYS    12902   1-01-80  12:00a
```

```
COMMAND  COM     7035   1-01-80  12:00a
ASM      COM    17945   1-01-80  12:00a
EDIT     COM     5410   1-01-80  12:00a
HELLO    ASM      299   1-01-80  12:00a
HELLO    COM       31   1-01-80  12:00a
```

WARNING: Do not change a diskette while a program is using it. A file that is still open is written to whatever diskette is in the drive.

Caring for Diskettes

- Keep diskettes away from magnets, heat and direct sunlight.
- Do not open the metal shutter or touch the surface of the diskette.
- Remove diskettes from the drives only when the drive light is out.
- Keep a copy of every diskette that holds work you cannot do again.

Volume Labels

A diskette may have a name of up to 11 characters, its *volume label*, written when it is formatted. DIR and VOL display it. Use labels to tell your diskettes apart.

This page intentionally left blank.

4

COS Commands

CONTENTS

About the Commands 4-2
CD (Change Directory) 4-2
CLS (Clear Screen) 4-3
COPY 4-3
DATE 4-4
DEL (Delete) 4-5
DIR (Directory) 4-6
ECHO 4-7
EXIT 4-7
MD (Make Directory) 4-7
RD (Remove Directory) 4-8
PAUSE 4-9
REM (Remark) 4-9
REN (Rename) 4-9
SET 4-10
TIME 4-11
TYPE 4-11
VER (Version) 4-12
VOL (Volume) 4-12
d: (Change the Default Drive) 4-12
Running Programs 4-12

About the Commands

All the commands in this chapter are built into COMMAND.COM: they are always available, whatever diskette is in the drive. Anything else you type is taken as the name of a program to run (see "Running Programs" at the end of this chapter).

Each command is described in the same way:

Purpose	What the command does.
Format	How to type it. See "Notation" in the Preface.
Remarks	How it works, and the details you need to know.
Example	The command at work, as it appears on the display.

Separate a command from its parameters with a space. A command line can hold up to 127 characters.

CD (Change Directory)

ALSO CHDIR

PURPOSE

Changes the current directory of a drive, or displays it.

FORMAT

CD [d:][*path*]

REMARKS

CD *path* makes that directory the current directory of its drive (the default drive if none is given). The default drive does not change: CD B:\DATA changes the current directory of drive B:, which DIR B: and B:*name* then use.

CD \ goes back to the root directory, and CD .. to the directory above. CD alone, or CD d:, displays the current directory of that drive. A directory that does not exist gives "Invalid directory".

EXAMPLE

```
A:\>md games

A:\>cd games

A:\GAMES>cd
A:\GAMES

A:\GAMES>cd ..

A:\>cd \games

A:\GAMES>cd b:
B:\

A:\GAMES>
```


CLS (Clear Screen)

PURPOSE

Clears the display.

FORMAT

CLS

REMARKS

The display is cleared and the prompt appears in the top left corner.

COPY

PURPOSE

Copies one or more files, to the same diskette with another name or to another diskette.

FORMAT

COPY [d:]*filename*[.*ext*] [d:][*filename*[.*ext*]]

REMARKS

The first file specification is the *source*; it may contain global characters. The second is the *target*:

- If the target is only a drive, B:, the copies get the same names on that drive.
- If the target is a name, the copy gets that name. With global characters in the target, each ? or * keeps the corresponding characters of the source name: COPY *.TXT B:*.BAK.
- If there is no target, the copy goes to the default drive with the same name.

When the source has global characters, COPY displays the name of each file as it copies it. At the end it tells you how many files it copied. A file that already exists on the target diskette is replaced.

The copy is exact, byte for byte, and keeps the date and time of the original. A file cannot be copied onto itself. If the target diskette fills up, the incomplete copy is erased and COPY says "Insufficient disk space".

EXAMPLE

```
A:\>copy letter.txt b:note.txt
        1 File(s) copied

A:\>copy *.com b:
A:COMMAND.COM
A:ASM.COM
A:EDIT.COM
A:HELLO.COM
A:NUMBERS.COM
A:FACTOR.COM
A:SORT.COM
        7 File(s) copied
```

DATE

PURPOSE

Displays and sets the date.

FORMAT

DATE [*mm-dd-yy*]

REMARKS

Without a parameter, DATE displays the date, with the day of the week, and asks for a new one. Press **Enter** to keep it.

Type the month (1-12), the day (1-31) and the year, separated by hyphens (-), slashes (/) or periods (.). The year may be given with two digits, 80 to 99 for 1980 to 1999 and 00 to 79 for 2000 to 2079, or with four, from 1980 to 2099. COS knows how many days each month has, leap years included. If the date is not valid, COS says "Invalid date" and asks again.

COS advances the date at midnight while the computer is on.

EXAMPLE

```
A:\>date
Current date is Tue  1-01-1980
Enter new date (mm-dd-yy): 10-14-81

A:\>date
Current date is Wed 10-14-1981
Enter new date (mm-dd-yy):
```

DEL (Delete)

ALSO ERASE

PURPOSE

Erases files.

FORMAT

DEL [d:]*filename*[.*ext*]

REMARKS

The file specification may contain global characters to erase several files. To erase every file on a diskette, type DEL *.*; because this cannot be undone, COS asks you first:

```
B:\>del *.*
All files in directory will be deleted!
Are you sure (Y/N)?y
```

Type Y and press Enter to go on; anything else cancels the command. Files marked read-only are not erased ("Access denied").

EXAMPLE

```
B:\>del letter.txt

B:\>
```

DIR (Directory)

PURPOSE

Lists the files on a diskette.

FORMAT

DIR [d:][*filename*[.*ext*]] [/W]

REMARKS

DIR displays the volume label of the diskette, then one line for each file: name, extension, size in bytes, and the date and time it was last written. The last line gives the number of files listed and the free space on the diskette.

Without a file specification, DIR lists every file. With one, it lists only the files that match; global characters are allowed. A filename without an extension means any extension: DIR LETTER lists LETTER.TXT, LETTER.OLD and so on.

The /W (wide) parameter lists only the names, five to a line.

Hidden and system files are not listed.

EXAMPLE

```
A:\>dir *.txt

 Volume in drive A is COS
 Directory of  A:\

README   TXT       94   1-01-80  12:00a
LETTER   TXT       47   1-01-80  12:00a
        2 File(s)   1409536 bytes free

A:\>dir /w

 Volume in drive A is COS
 Directory of  A:\

COS      SYS    COMMAND  COM    ASM      COM    EDIT     COM    HELLO    ASM
HELLO    COM    NUMBERS  COM    FACTOR   COM    SORT     COM    README   TXT
LETTER   TXT
       11 File(s)   1409536 bytes free
```

ECHO

PURPOSE

Displays a message.

FORMAT

ECHO [*message*]
ECHO [ON | OFF]

REMARKS

ECHO displays the message exactly as you typed it.

In a batch file, ECHO OFF stops COS from showing each command before it runs it, and ECHO ON shows them again. Without a message, ECHO displays "ECHO is on" or "ECHO is off".

EXAMPLE

```
A:\>echo Hello there
Hello there
```

EXIT

PURPOSE

Ends a command processor started by a program.

FORMAT

EXIT

REMARKS

The command processor started by COS is permanent, and EXIT does nothing in it.

MD (Make Directory)

ALSO MKDIR

PURPOSE

Makes a new directory.

FORMAT

MD [d:]*path*

REMARKS

The last name of the path is the new directory; the directories before it must exist. MD says "Unable to create directory" if a file or directory already has that name, if the directory above does not exist, or if there is no room for it (the root directory is full, or the diskette is full or write-protected).

EXAMPLE

```
A:\>md games

A:\>dir

 Volume in drive A is COS
 Directory of  A:\

COS      SYS    12902   1-01-80  12:00a
COMMAND  COM     7035   1-01-80  12:00a
ASM      COM    17945   1-01-80  12:00a
EDIT     COM     5410   1-01-80  12:00a
HELLO    ASM      299   1-01-80  12:00a
HELLO    COM       31   1-01-80  12:00a
NUMBERS  COM       69   1-01-80  12:00a
FACTOR   COM       81   1-01-80  12:00a
SORT     COM      124   1-01-80  12:00a
README   TXT       94   1-01-80  12:00a
LETTER   TXT       47   1-01-80  12:00a
UNSUP    COM       31   1-01-80  12:00a
GAMES          <DIR>     1-01-80  12:00a
       13 File(s)   1409536 bytes free

A:\>
```

RD (Remove Directory)

ALSO RMDIR

PURPOSE

Removes a directory.

FORMAT

RD [d:]*path*

REMARKS

The directory must be empty (only . and .. in it), and it cannot be the current directory of its drive. Otherwise, or if there is no such directory, RD says "Invalid path, not directory, or directory not empty". Erase the files in it first with DEL, and remove the directories in it with RD.

EXAMPLE

```
A:\>md games

A:\>copy hello.com games
        1 File(s) copied

A:\>rd games
Invalid path, not directory,
or directory not empty

A:\>del games\hello.com

A:\>rd games

A:\>
```


PAUSE

PURPOSE

Waits for a key.

FORMAT

PAUSE

REMARKS

PAUSE displays "Strike a key when ready . . ." and waits until a key is pressed. Use it, for example, to give yourself time to change a diskette.

REM (Remark)

PURPOSE

Does nothing.

FORMAT

REM [*remark*]

REN (Rename)

ALSO RENAME

PURPOSE

Changes the name of files.

FORMAT

REN [d:]*filename*[*.ext*] *filename*[*.ext*]

REMARKS

The first name is the file's present name; it may include a drive and global characters. The second is the new name, without a drive: the file stays on the same diskette. Global characters in the new name keep the corresponding characters of the old one, so REN *.TXT *.BAK changes the extension of every .TXT file.

If there is no such file, or a file with the new name already exists, COS says "Duplicate file name or file not found".

EXAMPLE

```
B:\>ren letter.txt letter.old

B:\>ren *.txt *.bak

B:\>dir

 Volume in drive B is WORK
 Directory of  B:\

README   BAK       94   1-01-80  12:00a
LETTER   OLD       47   1-01-80  12:00a
        2 File(s)   1456640 bytes free
```

SET

PURPOSE

Displays the environment.

FORMAT

SET

REMARKS

The environment is a list of settings in the form *name=value* that COS gives to every program it runs. COMSPEC tells programs where the command processor is. COS Version 1.00 does not change the environment: SET with parameters says "Changing the environment is not supported by COS".

EXAMPLE

```
A:\>set
COMSPEC=A:\COMMAND.COM
```

TIME

PURPOSE

Displays and sets the time.

FORMAT

TIME [*hh*[:*mm*[:*ss*[.*xx*]]]]

REMARKS

Without a parameter, TIME displays the time, in hours, minutes, seconds and hundredths of a second, and asks for a new one. Press **Enter** to keep it.

Type the hour (0-23) and, if you wish, the minutes (0-59), the seconds (0-59) and the hundredths (0-99). Parts you leave out are set to zero. If the time is not valid, COS says "Invalid time" and asks again.

EXAMPLE

```
A:\>time
Current time is  0:00:00.32
Enter new time: 9:30

A:\>time
Current time is  9:30:00.12
Enter new time:
```

TYPE

PURPOSE

Displays the contents of a file.

FORMAT

TYPE [d:]*filename*[.*ext*]

REMARKS

TYPE is for text files. It stops at the end of the file or at an end-of-file mark (Ctrl-Z, 1Ah). Program files and other binary files display as strange characters. Global characters are not allowed.

EXAMPLE

```
A:\>type readme.txt
Welcome to COS, the CucsiCorp Operating System.
Type DIR to see the files on this diskette.
```

VER (Version)

PURPOSE

Displays the version of COS.

FORMAT

VER

EXAMPLE

```
A:\>ver

CucsiCorp COS Version 1.00
```


VOL (Volume)

PURPOSE

Displays the volume label of a diskette.

FORMAT

VOL [d:]

EXAMPLE

```
A:\>vol b:

 Volume in drive B is WORK
```

d: (Change the Default Drive)

PURPOSE

Makes another drive the default drive.

FORMAT

d:

REMARKS

Type the drive letter and a colon. The prompt shows the new default drive. If the computer has no such drive, COS says "Invalid drive specification".

Running Programs

A word at the prompt that is not a command is the name of a program:

> [d:][*path*]*filename*[*.ext*] [*parameters*]

COS looks in the directory you name, or in the current directory, for *filename*.COM and then for *filename*.EXE, and runs the first one it finds. If you type the extension, only that file is tried. When you give only a name and the program is not in the current directory, COS looks for it in the root directory of the drive as well, so the programs of the System Diskette (EDIT, ASM) can be used from any directory. The rest of the line, the *parameters*, is given to the program exactly as you typed it.

If there is no .COM or .EXE file with that name, COS looks for *filename*.BAT, a batch file, and runs it. If there is no such file either, COS says "Bad command or file name". If there is not enough memory for a program, "Program too big to fit in memory".

When the program ends, its memory is given back and the prompt returns. The files it left open are closed.

```
A:\>md games

A:\>copy hello.com games
        1 File(s) copied

A:\>games\hello
Hello from the 8086!

A:\>cd games

A:\GAMES>factor
1
1
2
6
24
120
720
5040
40320

A:\GAMES>
```

This page intentionally left blank.

5

Messages

CONTENTS

Batch Files 5-2
About the Messages 5-2
Disk Errors 5-2
Messages in Alphabetic Order 5-3

Batch Files

A batch file is a text file, with the extension .BAT, that holds commands, one per line. Typing its name runs the commands in order, as if you had typed them: commands of COS, programs, and the drive changes. You can write one with EDIT.

COS shows each command, after the prompt, before it runs it. A line that begins with @ is not shown, and ECHO OFF stops showing the lines that follow. A batch file that names another batch file ends there, and the other one runs. Ctrl-C stops a batch file.

```
@ECHO OFF
ECHO Starting the game
B:
SPACE
```


AUTOEXEC.BAT

When COS starts, it runs AUTOEXEC.BAT from the root directory of the diskette it started from, if there is one. Use it to start a program, or to set the date, every time the computer is turned on.

About the Messages

This chapter lists, in alphabetic order, the messages that COS and COMMAND.COM display, with their meaning and what to do. In the messages, *x* stands for a drive letter and *xx* for a hexadecimal number.

Disk Errors

When a diskette cannot be read or written, COS tries three times and then displays the problem and a question:

```
Not ready reading drive B
Abort, Retry, Fail?
```

Correct the problem if you can — insert a diskette, close the drive, open the write-protect tab — and answer with one of these letters:

R (Retry)	Try the operation again.
F (Fail)	Give up this operation. The program or command is told it failed and goes on; a command usually displays another message.
A (Abort)	End the program that was using the diskette. If it was a command, the command processor starts again and shows its title.

The first line names the problem:

Not ready	There is no diskette in the drive, or the drive is open.
Write protect error	The diskette is write-protected.
Data error	A sector could not be read correctly.
Sector not found	The diskette is not formatted, or is damaged.
Seek error	The drive could not find the track.
Unknown media type	The diskette is not a COS (FAT) diskette, or has a format COS does not know.
General failure	Another error.

Messages in Alphabetic Order

Message	Meaning and action
Access denied	The file is read-only, or the name is a device the computer does not have. Choose another file.
All files in directory will be deleted! Are you sure (Y/N)?	You typed DEL *.*. Type Y and Enter to erase every file, or N and Enter to keep them.
Bad command or file name	What you typed is not a command, and there is no program with that name on the drive. Check the spelling and the drive.
Bad or missing command interpreter	COS could not find COMMAND.COM on the diskette it started from. Start the computer with a System Diskette.
Changing the environment is not supported by COS	SET only displays the environment.
COS: CALL 5 (CP/M entry) is not supported	The program uses the old CP/M way of calling the operating system. COS ended it.

Message	Meaning and action
COS: INT 21h function AH=*xx*h is not supported	The program asked COS for a service that COS Version 1.00 does not provide. COS ended it rather than give it a wrong answer. Chapter 7 lists the functions COS provides.
COS: INT 25h (absolute disk read) is not supported **COS: INT 26h (absolute disk write) is not supported**	The program tried to read or write diskette sectors directly. COS ended it.
COS.SYS is missing	The diskette in drive A: has the COS boot record but not COS.SYS. Use another System Diskette and press any key.
Disk error	The boot record could not read the System Diskette. Press any key to try again.
Duplicate file name or file not found	REN found no file with the old name, or a file with the new name already exists.
File cannot be copied onto itself	The source and the target of COPY are the same file. Give the copy another name or another drive.
File creation error	COPY could not create the target file: the directory is full, the name belongs to a read-only file, or the diskette is write-protected.
File not found	There is no file with that name. Check the name and the drive with DIR.
General failure	A command could not read a file because of a disk error you answered with F.
Insufficient disk space	The target diskette is full. The incomplete copy was erased. Make room or use another diskette.
Invalid date	The date is not one COS can keep: check the month, the day and the year (1980-2099).
Invalid directory	CD, or the target of COPY, names a directory that does not exist.
Invalid drive specification	The computer does not have that drive.
Invalid number of parameters	REN needs two names.
Invalid path, not directory, or directory not empty	RD could not remove the directory: it does not exist, it is a file, it still has files or directories in it, or it is the current directory.

Message	Meaning and action
Invalid switch	DIR accepts only /W.
Invalid time	Hours go from 0 to 23, minutes and seconds from 0 to 59.
Press any key to restart	The boot record could not start COS (see the message before it). Insert a System Diskette and press a key.
Program too big to fit in memory	There is not enough memory for the program.
Required parameter missing	The command needs a file name: COPY, DEL and TYPE.
Strike a key when ready . . .	PAUSE is waiting for a key.
Unable to create directory	MD could not make the directory: the name is taken, the directory above does not exist, or there is no room.
^C	You pressed Ctrl-C. The command or program was stopped.

This page intentionally left blank.

6

Programming COS

CONTENTS

Introduction 6-2
Tools 6-2
Your First Program 6-2
.COM Programs 6-3
.EXE Programs 6-4
The Program Segment Prefix 6-5
The Environment 6-5
Memory 6-6
Files and Handles 6-6
Errors 6-7
Ctrl-C 6-7
Running Another Program 6-8
Terminate and Stay Resident 6-8
The Processor 6-9
Rules for Well-Behaved Programs 6-9

Introduction

This chapter is for programmers. It explains how programs are written for COS, how COS loads and runs them, and how a program asks COS for its services. It assumes you know the 8086 processor and its assembly language.

A program asks for a service by loading a function number into register AH, and the other parameters into other registers, and executing an INT 21h instruction. COS does the work and returns, with results in registers. When a function fails, COS sets the carry flag (CF) and puts an error code in AX. Chapter 7 describes every function; Appendix A lists the error codes.

Tools

COS programs are written in 8086 assembly language. Everything you need is on the System Diskette:

1. Write the program with the editor: **EDIT GAME.ASM** (Chapter 9).
2. Assemble it: **ASM GAME** makes GAME.COM (Chapter 10).
3. Run it: **GAME**.

COS itself is written in the same language, and ASM assembles it: Chapter 10 shows how. COS.SYS can be anywhere on the System Diskette; the boot record finds it in the directory.

Your First Program

This program displays a message and ends:

```
; hello.asm - the classic: print a string with COS service 09h
org 100h

    mov dx, message     ; DS:DX points to the text
    mov ah, 09h         ; "display a string ending in $"
    int 21h
    ret                 ; a .COM program may end with RET

message db 'Hello from the 8086!', 13, 10, '$'
```

Assembled, it is a .COM file of 31 bytes. Run it by typing HELLO.

Ending a Program

A program ends with one of these:

MOV AX,4C00h / INT 21h	The best way. AL is a return code (0 means success) that the parent can read with function 4Dh.
INT 20h	CS must point to the PSP. Return code 0.
RET	In a .COM program, with the stack as it was at the start: the word 0 at the top of the stack sends it to the INT 20h at the start of the PSP.

When a program ends, COS closes its files, frees its memory, restores interrupt vectors 22h, 23h and 24h, and returns to the program that started it.

.COM Programs

A .COM file is an image of the program: COS copies it into memory at offset 100h of a segment and jumps to it. The program must be assembled with ORG 100h and fit in 64 KB less 256 bytes for the PSP and the stack (the largest .COM file is 65,280 bytes).

At the start of a .COM program:

CS, DS, ES, SS	The segment of the program segment prefix (PSP).
IP	0100h.
SP	FFFEh, or the top of the memory given to the program if that is less than 64 KB. A word of 0 is on the stack.
AX	0000h. (BX = 0, CX = 00FFh, DX = PSP segment, SI = DI = BP = 0.)
Flags	Interrupts enabled.
Memory	The largest free block of memory belongs to the program.
DTA	PSP:0080h.

> **NOTE:** Because a .COM program owns the largest block of memory, it must give back what it does not need with function 4Ah before it can allocate memory with function 48h or run another program with function 4Bh.

.EXE Programs

An .EXE file begins with a header that tells COS how big the program is, where its stack and its first instruction are, and which words must be adjusted for the segment it is loaded at (the *relocation table*). An .EXE program may be larger than 64 KB and use several segments.

Offset	Contents
00h	'MZ', the signature
02h	Bytes in the last 512-byte page of the file (0 = the whole page)
04h	Pages in the file, header included
06h	Number of relocation entries
08h	Size of the header in paragraphs
0Ah	Minimum paragraphs needed above the program
0Ch	Maximum paragraphs wanted above the program
0Eh	SS, relative to the start of the program
10h	SP
12h	Checksum (not checked)
14h	IP
16h	CS, relative to the start of the program
18h	Offset of the relocation table in the file
1Ah	Overlay number (0)

Each relocation entry is two words, an offset and a segment relative to the start of the program. COS loads the program image (the file after the header) at the paragraph after the PSP, adds that segment to every word named in the relocation table, and starts the program with DS and ES pointing to the PSP, and CS:IP and SS:SP as given in the header. The program gets the memory it needs plus the maximum it asks for, as far as there is memory.

An assembler that writes flat binary files can write an .EXE file too, by writing the header itself with DB and DW. The relocation entries point at words such as the operand of MOV AX,*segment*.

The Program Segment Prefix

COS builds a 256-byte *program segment prefix* (PSP) in front of every program. A program finds its PSP in DS and ES when it starts, or with function 62h.

Offset	Contents
00h	INT 20h instruction
02h	Segment just past the memory given to the program
05h	CALL to the CP/M entry (not supported)
0Ah	Terminate address: where the parent continues (INT 22h)
0Eh	Ctrl-C handler address when the program started (INT 23h)
12h	Critical error handler address when the program started (INT 24h)
16h	Segment of the parent's PSP
18h	Handle table: 20 bytes, FFh for an unused handle
2Ch	Segment of the environment
2Eh	Used by COS: the program's stack while it runs a child
32h	Number of entries in the handle table (20)
34h	Address of the handle table
50h	INT 21h and RETF instructions
5Ch	First parameter as an unopened file control block
6Ch	Second parameter as an unopened file control block
80h	Length of the command tail
81h	Command tail: the parameters as typed, ending with a carriage return (0Dh)

The command tail of **child One two** is a length of 8 and the characters " One two" followed by 0Dh. The tail area is also the default disk transfer area (DTA), so read the parameters before you use functions that write to the DTA, such as 4Eh.

The Environment

The environment is a block of memory of strings in the form *NAME=value*, each ending with a zero byte, with an extra zero after the last one. A word 0001h and the full name of the program, such as A:\GAME.COM, follow it. The segment of the environment is at offset 2Ch of the PSP. Under COS it holds:

```
COMSPEC=A:\COMMAND.COM
```

Memory

The memory COS does not use itself is divided into *blocks*, each preceded by a 16-byte *memory control block* (MCB): a byte 'M' (or 'Z' for the last block), the segment of the PSP that owns the block (0 if it is free), and the size of the block in paragraphs. A program allocates, resizes and frees blocks with functions 48h, 4Ah and 49h; COS gives the first free block that is large enough. All the blocks a program owns are freed when it ends, unless it ends with function 31h (terminate and stay resident).

With COMMAND.COM loaded, the largest block a program can have is 91E2h paragraphs: 597,536 bytes. Chapter 8 shows how memory is used.

Files and Handles

A program uses files through *handles*: small numbers that COS gives it when it opens or creates a file (functions 3Dh and 3Ch), and that it then uses to read (3Fh), write (40h), move (42h) and close (3Eh) the file. Every program starts with three handles open:

Handle	Name	Device
0	Standard input	CON (the keyboard)
1	Standard output	CON (the display)
2	Standard error	CON (the display)

A program may have 20 handles open at once, and COS 20 files for all programs together. A program started with function 4Bh inherits its parent's open handles, so it writes to the same standard output.

Reading from the console through handle 0 is *cooked*: COS reads a whole line with editing, as function 0Ah does, and returns it followed by a carriage return and a line feed. Function 4401h can set the console to *raw* mode, in which each key is returned as it is pressed, without echo.

COS writes a file's data to the diskette when the program closes the file (3Eh) or commits it (68h), at disk reset (0Dh), when the program ends, and whenever it needs to read another track of the diskette. A program that keeps a file open for a long time should commit it from time to time.

Example: Writing a File

```
    mov ah, 3Ch             ; create SCORES.DAT
    mov dx, name
    xor cx, cx              ; normal attributes
    int 21h
    jc error                ; CF set: AX = error code
    mov bx, ax              ; the handle
    mov ah, 40h             ; write 4 bytes
    mov dx, scores
    mov cx, 4
    int 21h
    jc error
    mov ah, 3Eh             ; close: the data reaches the diskette
    int 21h
    ...
name   db 'SCORES.DAT', 0
scores db 10, 20, 30, 40
```


Errors

Handle functions and most other functions report an error by setting the carry flag and returning an error code in AX: always test CF after INT 21h. Function 59h returns the code of the last error again. Appendix A lists the codes.

Disk errors are reported to the user first, by the critical error handler, which asks "Abort, Retry, Fail?". If the user answers F, the function returns error 83 (53h). A program can install its own critical error handler at INT 24h; Chapter 8 describes how it is called.

Ctrl-C

When the user presses Ctrl-C while a program waits in a console input function (01h, 08h, 0Ah, 0Bh, or a cooked read of handle 0), COS displays ^C and executes INT 23h. The default handler ends the program. A program may install its own handler with function 25h:

- If the handler returns with IRET, COS carries out the function again.
- The handler may also end the program, or jump to a safe place in it after resetting its stack.

COS restores INT 23h when the program ends. Functions 06h and 07h do not check for Ctrl-C; a program that reads keys with them sees Ctrl-C as the character 03h.

Running Another Program

Function 4Bh, EXEC, loads and runs another program and returns when it ends. The caller passes the program's name and a parameter block:

```
    mov ah, 4Ah             ; first, give back unused memory
    mov bx, (program_end - $$ + 100h + 15) / 16
    int 21h
    mov dx, child           ; DS:DX = name
    mov bx, block           ; ES:BX = parameter block
    mov [block+4], cs
    mov [block+8], cs
    mov [block+12], cs
    mov ax, 4B00h
    int 21h
    jc failed
    mov ah, 4Dh             ; the child's return code
    int 21h
    ...
child db 'CHILD.COM', 0
tail  db 7, ' hello!', 13
block dw 0                  ; environment: a copy of ours
      dw tail, 0            ; command tail
      dw 5Ch, 0             ; first FCB
      dw 6Ch, 0             ; second FCB
```

All registers are kept across the call. The DTA is set to the caller's PSP:0080h.

Terminate and Stay Resident

A program can end and leave part of itself in memory, for example an interrupt handler, with function 31h (or INT 27h). DX gives the number of paragraphs to keep, counted from the PSP; the rest is freed. Open files are not closed.

The Processor

The pc86 computer's 8086 executes every 8086 instruction except these, which stop the computer with an error instead of doing something different: the decimal adjust instructions (DAA, DAS, AAA, AAS, AAM, AAD), XLAT, LOCK, WAIT, ESC, INTO, and undocumented instructions. Avoid them in COS programs.

Rules for Well-Behaved Programs

- Use the COS functions for the keyboard, the display and files; use the BIOS (Appendix D) for what COS does not do, such as moving the cursor or graphics.
- Test the carry flag after every function that can fail.
- Close every file you open, and end with function 4Ch.
- Restore any interrupt vector you change, except 22h, 23h and 24h, which COS restores for you.
- Do not call INT 21h from an interrupt handler or from a critical error handler: COS does not allow a function to be interrupted by another.

This page intentionally left blank.

7

Interrupt and Function Reference

CONTENTS

The COS Interrupts 7-2
Calling Conventions 7-2
Functions in Numeric Order 7-2
Console Functions 7-4
Disk and Directory Functions 7-6
File Functions 7-8
Program Functions 7-12
Memory Functions 7-13
Other Functions 7-14

The COS Interrupts

INT	Use
20h	Terminate the program (CS = PSP). Use function 4Ch instead.
21h	Function request: the functions in this chapter.
22h	Terminate address. Not called; COS keeps it in the PSP and restores it.
23h	Ctrl-C handler. Called by COS; a program may replace it.
24h	Critical error handler. Called by COS; a program may replace it.
25h, 26h	Absolute disk read and write: not supported. The program is ended with a message.
27h	Terminate and stay resident, DX = bytes to keep from the PSP. Use function 31h instead.
28h	Idle: COS calls nothing; returns at once.
29h	Fast console output of AL.
2Ah, 2Fh	Network and multiplex: nothing is installed. They return at once with AL unchanged.

Calling Conventions

Load AH with the function number and the other registers as shown, and execute INT 21h. COS preserves all registers except those that return a result. Functions that can fail return with the carry flag clear on success, or set with an error code in AX. "ASCIIZ" means a string ending with a zero byte. File names may include a drive and a path: B:\GAMES\SCORES.DAT, ..\NOTES.TXT (Chapter 3). A path may have up to 63 characters.

Functions in Numeric Order

AH	Function	AH	Function
00h	Terminate program	38h	Country information
01h	Keyboard input with echo	39h	Make a directory
		3Ah	Remove a directory
		3Bh	Change the current directory
02h	Display output	3Ch	Create a file
06h	Direct console I/O	3Dh	Open a file

AH	Function	AH	Function
07h	Direct keyboard input	3Eh	Close a handle
08h	Keyboard input, no echo	3Fh	Read from a file or device
09h	Display a string	40h	Write to a file or device
0Ah	Buffered keyboard input	41h	Delete a file
0Bh	Check keyboard status	42h	Move the file pointer
0Ch	Clear buffer, then input	43h	File attributes
0Dh	Disk reset	44h	I/O control (IOCTL)
0Eh	Select disk	45h	Duplicate a handle
19h	Current disk	46h	Force a duplicate handle
1Ah	Set the disk transfer address	47h	Current directory
25h	Set an interrupt vector	48h	Allocate memory
29h	Parse a file name	49h	Free memory
2Ah	Get the date	4Ah	Resize a memory block
2Bh	Set the date	4Bh	Load and execute a program
2Ch	Get the time	4Ch	Terminate with a return code
2Dh	Set the time	4Dh	Return code of a child
2Eh	Set the verify switch	4Eh	Find the first file
2Fh	Get the disk transfer address	4Fh	Find the next file
30h	COS version number	50h	Set the PSP
31h	Terminate and stay resident	51h, 62h	Get the PSP
33h	Ctrl-Break check, boot drive	54h	Get the verify switch
34h	Address of the InDOS flag	56h	Rename or move a file
35h	Get an interrupt vector	57h	File date and time
36h	Free disk space	58h	Allocation strategy
37h	Switch character	59h	Extended error
		5Bh	Create a new file
		68h	Commit a file

Functions not in this list — among them 03h-05h (auxiliary and printer), 0Fh-18h and 21h-24h (file control blocks), 1Bh and 1Ch (FAT information), 26h-28h, 32h, 52h and 5Ah-61h — are not supported: COS displays "COS: INT 21h function AH=*xx*h is not supported" and ends the program.

Console Functions

00h Terminate Program

ENTRY AH = 00h

Ends the program with return code 0. Same as function 4Ch with AL = 0.

01h Keyboard Input with Echo

ENTRY AH = 01h

RETURN AL = character

Waits for a key and displays it. For a key with no character (a function or cursor key) AL = 0, and the next call returns its extended code (Appendix B). Ctrl-C runs INT 23h.

02h Display Output

ENTRY AH = 02h
DL = character

Displays the character. Carriage return (0Dh), line feed (0Ah), backspace (08h) and bell (07h) act on the cursor; tab (09h) moves to the next multiple of eight columns.

06h Direct Console I/O

ENTRY AH = 06h
DL = FFh to read, or a character to display

RETURN For DL = FFh: ZF clear and AL = character if a key was waiting; ZF set and AL = 0 if not

Does not wait and does not check for Ctrl-C.

07h Direct Keyboard Input

ENTRY AH = 07h

RETURN AL = character

Waits for a key; no echo, no Ctrl-C check.

08h Keyboard Input without Echo

ENTRY AH = 08h

RETURN AL = character

As 07h, but Ctrl-C runs INT 23h.

09h Display a String

ENTRY AH = 09h
DS:DX = string ending with '$'

Displays the characters up to, not including, the dollar sign.

0Ah Buffered Keyboard Input

ENTRY AH = 0Ah
DS:DX = buffer: byte 0 = its size, including the carriage return

RETURN Byte 1 = number of characters typed; then the characters and a carriage return

Reads a line with the editing keys of Chapter 2: Backspace, Esc and Enter. Control characters are stored and displayed as ^*X*; when the buffer is full the bell sounds. The carriage return is echoed, the line feed is not. Ctrl-C runs INT 23h.

0Bh Check Keyboard Status

ENTRY AH = 0Bh

RETURN AL = FFh if a key is waiting, 00h if not

A waiting Ctrl-C runs INT 23h.

0Ch Clear Keyboard Buffer and Input

ENTRY AH = 0Ch
AL = 01h, 06h, 07h, 08h or 0Ah: the input function to run then other registers as for that function

Throws away the keys typed ahead, then carries out the function in AL.

Disk and Directory Functions

0Dh Disk Reset

ENTRY AH = 0Dh

Writes to the diskettes everything COS keeps in memory for them. Files stay open.

0Eh Select Disk

ENTRY AH = 0Eh
DL = drive (0 = A:, 1 = B:)

RETURN AL = number of drives

Makes the drive the default drive, if it exists.

19h Current Disk

ENTRY AH = 19h

RETURN AL = default drive (0 = A:, 1 = B:)

1Ah Set the Disk Transfer Address

ENTRY AH = 1Ah
DS:DX = DTA

The DTA receives the results of functions 4Eh and 4Fh. When a program starts it is at PSP:0080h.

2Fh Get the Disk Transfer Address

ENTRY AH = 2Fh

RETURN ES:BX = DTA

36h Free Disk Space

ENTRY AH = 36h
DL = drive (0 = default, 1 = A:, 2 = B:)

RETURN AX = sectors per allocation unit, or FFFFh if the drive is not valid
BX = free allocation units
CX = bytes per sector (512)
DX = allocation units on the diskette

Free bytes = AX × BX × CX.

39h Make a Directory

ENTRY AH = 39h
DS:DX = ASCIIZ path of the new directory

RETURN CF set: AX = 3 (a directory of the path does not exist) or 5 (the name exists, or there is no room)

The new directory gets its . and .. entries. A full root directory cannot grow; a full subdirectory gets another allocation unit.

3Ah Remove a Directory

ENTRY AH = 3Ah
DS:DX = ASCIIZ path

RETURN CF set: AX = 3 (no such directory), 5 (not empty, or read-only) or 10h (the current directory of its drive)

3Bh Change the Current Directory

ENTRY AH = 3Bh
DS:DX = ASCIIZ path

RETURN CF set: AX = 3 if the path is not a directory

Changes the current directory of the drive in the path (the default drive if there is none); the default drive stays the same.

47h Current Directory

ENTRY AH = 47h
DL = drive (0 = default, 1 = A:, 2 = B:)
DS:SI = 64-byte buffer

RETURN The buffer holds the path of the current directory, without the drive and the first backslash: "GAMES\LEVELS", or an empty string for the root. CF set, AX = 0Fh for a drive that does not exist

4Eh Find the First File

ENTRY AH = 4Eh
DS:DX = ASCIIZ file specification, global characters allowed
CX = attributes to include

RETURN CF clear: the DTA describes the file; CF set: AX = 2 (no file) or 3 (bad path)

The search is in the directory of the file specification. Normal and read-only files are always found. Hidden (02h), system (04h) and directory (10h) entries are found only if their bits are in CX; with 10h, the . and .. entries of a subdirectory are found too. With CX = 08h, only the volume label is found (it is in the root directory). The DTA receives:

Offset	Contents
00h-14h	Used by COS for function 4Fh: do not change
15h	Attribute of the file
16h	Time of the file
18h	Date of the file
1Ah	Size of the file (doubleword)
1Eh	Name and extension as ASCIIZ, e.g. "HELLO.COM"

4Fh Find the Next File

ENTRY AH = 4Fh
DTA as left by 4Eh or 4Fh

RETURN CF clear: the DTA describes the next file; CF set: AX = 12h, no more files

File Functions

3Ch Create a File

ENTRY AH = 3Ch
DS:DX = ASCIIZ file name
CX = attributes (00h normal, 01h read-only, 02h hidden, 04h system, 20h archive)

RETURN CF clear: AX = handle, open for reading and writing
CF set: AX = 3, 4 or 5

Creates the file, or empties it if it exists (unless it is read-only). The directory entry is written to the diskette at once, so a full directory or a write-protected diskette is reported here.

3Dh Open a File

ENTRY
```
AH = 3Dh
DS:DX = ASCIIZ file name
AL = 0 read, 1 write, 2 read and write
```

RETURN
```
CF clear: AX = handle
CF set: AX = 2, 4, 5 or 0Ch
```

The file pointer is at the start of the file. The names CON and NUL open the devices.

3Eh Close a Handle

ENTRY
```
AH = 3Eh
BX = handle
```

RETURN
```
CF set: AX = 6 (invalid handle) or a disk error
```

When the last handle to a file is closed, its directory entry (size, date, time) is updated and its data written to the diskette.

3Fh Read from a File or Device

ENTRY
```
AH = 3Fh
BX = handle
CX = bytes to read
DS:DX = buffer
```

RETURN
```
CF clear: AX = bytes read (0 at the end of the file)
CF set: AX = 5 or 6
```

Reads from the file pointer on, and moves the pointer past what was read. From the console, see "Files and Handles" in Chapter 6.

40h Write to a File or Device

ENTRY
```
AH = 40h
BX = handle
CX = bytes to write
DS:DX = data
```

RETURN
```
CF clear: AX = bytes written
CF set: AX = 5 or 6
```

If the diskette fills up, fewer bytes are written and AX says how many; this is not an error. With CX = 0 the file is cut off at the file pointer.

41h Delete a File

ENTRY AH = 41h
DS:DX = ASCIIZ file name, no global characters

RETURN CF set: AX = 2 or 5

42h Move the File Pointer

ENTRY AH = 42h
BX = handle
CX:DX = distance (signed)
AL = 0 from the start, 1 from the current position, 2 from the end

RETURN CF clear: DX:AX = new position
CF set: AX = 1 or 6

AL = 2 with CX:DX = 0 returns the size of the file.

43h File Attributes

ENTRY AH = 43h
DS:DX = ASCIIZ file name
AL = 0 to get, 1 to set them from CX

RETURN CF clear: CX = attributes (AL = 0)
CF set: AX = 1, 2 or 5

The volume label and directory bits cannot be set.

45h Duplicate a Handle

ENTRY AH = 45h
BX = handle

RETURN CF clear: AX = new handle to the same file and pointer
CF set: AX = 4 or 6

46h Force a Duplicate Handle

ENTRY AH = 46h
BX = handle
CX = second handle

RETURN CF set: AX = 6

Closes CX if it is open, and makes it refer to the same file as BX.

56h Rename or Move a File

ENTRY
```
AH = 56h
DS:DX = ASCIIZ present name
ES:DI = ASCIIZ new name
```
RETURN
```
CF set: AX = 2, 3, 5 or 11h
```

Both names must be on the same drive (error 11h). If a file with the new name exists, error 5. If the new name is in another directory, the file moves there. A directory can be renamed, but not moved.

57h File Date and Time

ENTRY
```
AH = 57h
BX = handle
AL = 0 to get; 1 to set from CX (time) and DX (date)
```
RETURN
```
CF clear: CX = time, DX = date (AL = 0)
CF set: AX = 1 or 6
```

A date and time that are set are recorded when the file is closed, instead of the current ones. Chapter 8 shows their format.

5Bh Create a New File

ENTRY As 3Ch

RETURN As 3Ch; CF set, AX = 50h if the file exists

68h Commit a File

ENTRY
```
AH = 68h
BX = handle
```
RETURN
```
CF set: AX = 6 or a disk error
```

Writes the file's data and directory entry to the diskette, as closing it would, and leaves it open.

44h I/O Control (IOCTL)

ENTRY
```
AH = 44h
AL = subfunction
BX = handle (00h, 01h, 06h, 07h) or BL = drive, 0 = default
(08h, 09h, 0Eh)
```

AL	Subfunction
00h	Get device information in DX. Devices: bit 7 set; CON 80D3h (bit 5 set in raw mode), NUL 8084h. Files: drive number in bits 0-5, bit 6 set if not written to.
01h	Set device information from DX (DH = 0); only bit 5, raw mode, of CON is used.
06h	Input status: AL = FFh ready, 00h not ready (a file at its end; the keyboard with no key).
07h	Output status: always AL = FFh.
08h	Removable media: AX = 0, diskettes are removable.
09h	Remote drive: DX = 0, local.
0Eh	Logical drive map: AL = 0, one letter per drive.

Program Functions

31h Terminate and Stay Resident

ENTRY
AH = 31h
AL = return code
DX = paragraphs to keep, from the PSP

Ends the program but leaves the first DX paragraphs of its memory in place, and its files open.

4Bh Load and Execute a Program (EXEC)

ENTRY
AX = 4B00h
DS:DX = ASCIIZ program name, with its extension
ES:BX = parameter block

RETURN
CF clear: the program ran and ended
CF set: AX = 2, 8 or 0Bh

Offset	Parameter block
00h	Segment of the environment to give it, or 0 for a copy of the caller's
02h	Address (offset, segment) of the command tail
06h	Address of the first file control block, copied to PSP:5Ch
0Ah	Address of the second file control block, copied to PSP:6Ch

Loads a .COM or .EXE program, as described in Chapter 6, and runs it; the call returns when it ends, with all registers as they were. AL = 01h (load only) and 03h (overlay) are not supported.

4Ch Terminate with a Return Code

ENTRY
```
AH = 4Ch
AL = return code
```

4Dh Return Code of a Child

ENTRY
```
AH = 4Dh
```

RETURN
```
AL = return code of the last program that ended
AH = how it ended: 0 normally, 1 by Ctrl-C, 2 by a critical
error (Abort), 3 resident
```

The code can be read only once.

50h Set the PSP

ENTRY
```
AH = 50h
BX = segment of the PSP to make current
```

62h Get the PSP

ENTRY
```
AH = 62h (or 51h)
```

RETURN
```
BX = segment of the current PSP
```

Memory Functions

48h Allocate Memory

ENTRY
```
AH = 48h
BX = paragraphs
```

RETURN
```
CF clear: AX = segment of the block
CF set: AX = 7 or 8, BX = size of the largest free block
```

Asking for FFFFh paragraphs always fails and tells you how much memory there is.

49h Free Memory

ENTRY
```
AH = 49h
ES = segment of the block
```

RETURN
```
CF set: AX = 9
```

4Ah Resize a Memory Block

ENTRY
```
AH = 4Ah
ES = segment of the block
BX = new size in paragraphs
```

RETURN
```
CF set: AX = 8 or 9; BX = largest size possible
```

58h Allocation Strategy

ENTRY
```
AX = 5800h to get, 5801h to set from BX
```

RETURN
```
AX = 0: first fit
```

COS uses first fit only; setting another strategy is not supported.

Other Functions

25h Set an Interrupt Vector

ENTRY
```
AH = 25h
AL = interrupt number
DS:DX = handler
```

35h Get an Interrupt Vector

ENTRY
```
AH = 35h
AL = interrupt number
```

RETURN
```
ES:BX = handler
```

29h Parse a File Name

ENTRY
```
AH = 29h
DS:SI = text to parse
ES:DI = 37-byte file control block
AL = options: bit 0 skip leading separators; bit 1 keep the
FCB's drive if none is given; bit 2 keep its filename; bit 3
keep its extension
```

RETURN
```
AL = 00h, 01h if there were global characters, FFh if the
drive is not valid
DS:SI = the first character after the name
ES:DI+0 = drive (0 = default, 1 = A:); +1 = filename, 8
characters; +9 = extension, 3 characters
```

An * fills the rest of the field with ?. COMMAND.COM uses this function to fill the FCBs in the PSP.

2Ah Get the Date

ENTRY AH = 2Ah

RETURN CX = year (1980-2099), DH = month, DL = day
AL = day of the week (0 = Sunday)

2Bh Set the Date

ENTRY AH = 2Bh
CX = year, DH = month, DL = day

RETURN AL = 00h if valid, FFh if not

2Ch Get the Time

ENTRY AH = 2Ch

RETURN CH = hour (0-23), CL = minutes
DH = seconds, DL = hundredths

The time comes from the timer tick (18.2 per second), so the hundredths advance in steps of about 5.

2Dh Set the Time

ENTRY AH = 2Dh
CH = hour, CL = minutes, DH = seconds, DL = hundredths

RETURN AL = 00h if valid, FFh if not

2Eh Set the Verify Switch

ENTRY AH = 2Eh
AL = 0 off, 1 on

The switch is kept for programs that read it back with function 54h.

30h COS Version Number

ENTRY AH = 30h

RETURN AL = 3, AH = 30: the 3.30 level of services
BH = FFh, BL = 0, CX = 0

Programs that check the version find the level of services they need. Use VER to see the COS version.

33h Ctrl-Break Check, Boot Drive

ENTRY AH = 33h
AL = 0 get, 1 set from DL, 5 get the boot drive

RETURN DL = switch (0 off, 1 on), or the boot drive (1 = A:)

COS checks for Ctrl-C on console input only, whatever the switch says.

34h Address of the InDOS Flag

ENTRY AH = 34h

RETURN ES:BX = a byte that is not 0 while a function is being carried out

37h Switch Character

ENTRY AX = 3700h

RETURN DL = '/', AL = 0

38h Country Information

ENTRY AX = 3800h
DS:DX = 34-byte buffer

RETURN BX = 1 (United States); the buffer holds the date format (0: month-day-year), the currency symbol "$", the separators ",", ".", "-" and ":", and the 12-hour clock

54h Get the Verify Switch

ENTRY AH = 54h

RETURN AL = 0 off, 1 on

59h Extended Error

ENTRY AH = 59h, BX = 0

RETURN AX = code of the last error; BH = class, BL = suggested action, CH = locus

8

Technical Information

CONTENTS

Memory Map 8-2
How COS Starts 8-2
Diskette Format 8-3
The File Allocation Table 8-4
Directory Entries 8-4
Disk Buffering 8-5
Critical Errors 8-5
System Limits 8-5
Compatibility 8-6

Memory Map

Address	Contents
0000:0000	Interrupt vectors
0040:0000	BIOS data area
0070:0000	COS.SYS: code and data (11 KB), its own PSP, the track buffer (9 KB), the file allocation tables of drives A: and B: (4.5 KB each) and its stacks
085Ch	First memory control block: the master environment
0860h	COMMAND.COM's environment, then COMMAND.COM (PSP at 0863h) with its 16 KB copy buffer and stack
0E09h	Free memory: programs (583 KB)
A000:0000	Video memory
F000:0000	BIOS read-only memory

The segments of COMMAND.COM and the free memory are those of COS Version 1.00 started from its System Diskette; a program should take them from its PSP and from the memory functions, not from this table.

How COS Starts

1. The BIOS reads sector 1 of the diskette in drive A: to 0000:7C00h and jumps to it with DL = 0.
2. The boot record reads the BIOS parameter block in itself, loads the first file allocation table at 0000:7E00h, finds COS.SYS in the root directory and loads it at 0070:0000h, following its chain of allocation units. Then it jumps to it, with DL still the boot drive.
3. COS.SYS counts the diskette drives (BIOS equipment word), sets interrupt vectors 20h to 2Fh, makes the memory control blocks up to the top of memory given by INT 12h, builds the master environment, and runs A:\COMMAND.COM with function 4Bh. If COMMAND.COM ever ends, COS runs it again.

Diskette Format

	1.44 MB	720 KB
Tracks per side	80	80
Sides	2	2
Sectors per track	18	9
Bytes per sector	512	512
Sectors per allocation unit	1	2
Reserved sectors (boot record)	1	1
File allocation tables	2 × 9 sectors	2 × 3 sectors
Root directory	224 entries, 14 sectors	112 entries, 7 sectors
Allocation units	2,847	713
Media descriptor	F0h	F9h

Sectors are numbered from 0, track by track: logical sector *n* is on track *n* / sectors-per-track / 2, side (*n* / sectors-per-track) mod 2, sector *n* mod sectors-per-track + 1. The boot record is sector 0, then come the file allocation tables, the root directory and the data area.

The BIOS Parameter Block

Bytes 0Bh to 1Dh of the boot record describe the diskette. COS reads them from every diskette it uses, so any diskette with these values works.

Offset	Size	Contents
00h	3	Jump to the boot code
0Bh	Word	Bytes per sector: 512
0Dh	Byte	Sectors per allocation unit
0Eh	Word	Reserved sectors
10h	Byte	Number of file allocation tables
11h	Word	Root directory entries
13h	Word	Total sectors
15h	Byte	Media descriptor
16h	Word	Sectors per file allocation table
18h	Word	Sectors per track

Offset	Size	Contents
1Ah	Word	Sides (heads)
1FEh	Word	Signature 55h AAh

The File Allocation Table

The file allocation table (FAT) has a 12-bit entry for every allocation unit, numbered from 2. An entry of 000h means the unit is free; FF8h to FFFh, that it is the last unit of a file; any other value, the number of the file's next unit. The entry of unit *n* is the word at byte *n* × 3 / 2 of the table: its low 12 bits if *n* is even, its high 12 bits if *n* is odd. COS keeps the whole table of each drive in memory and writes it to both copies on the diskette.

Directory Entries

Offset	Size	Contents
00h	8	Filename, padded with spaces. First byte 00h: end of the directory; E5h: an erased entry
08h	3	Extension, padded with spaces
0Bh	Byte	Attribute: 01h read-only, 02h hidden, 04h system, 08h volume label, 10h directory, 20h archive (set when the file is written)
0Ch	10	Reserved
16h	Word	Time
18h	Word	Date
1Ah	Word	First allocation unit (0 for an empty file)
1Ch	Dword	Size in bytes (0 for a directory)

Subdirectories

A subdirectory is kept like a file, in a chain of allocation units, and holds directory entries of 32 bytes; its own entry has attribute 10h and size 0. Its first two entries are . (first allocation unit: the subdirectory itself) and .. (the directory that holds it; 0 for the root directory). The root directory has a fixed size (Diskette Format, above); a subdirectory grows by one allocation unit, filled with zeros, whenever it is full.

Time and Date

```
Time:  bits 15-11 hours (0-23), 10-5 minutes, 4-0 seconds / 2
Date:  bits 15-9 year - 1980, 8-5 month (1-12), 4-0 day (1-31)
```

Disk Buffering

COS reads diskettes a whole track at a time into its track buffer: one turn of the diskette, a fifth of a second, brings in 18 sectors. Reads from the same track then need no diskette access.

Writes go into the buffer, and the sectors changed are written back together, in one operation, when COS needs another track, when a file is closed or committed, at disk reset, and when a program ends. A new directory entry (function 3Ch or 5Bh) and changes to the directory (41h, 43h, 56h) are written at once.

Before each use of a drive, COS asks the BIOS whether the diskette has been changed (INT 13h AH=16h). If it has, COS reads the new diskette's boot record and file allocation table.

Critical Errors

When a diskette operation still fails after three tries, COS calls INT 24h with:

AH	Bit 0: 0 reading, 1 writing. Bits 1-2: area (3 = data). Bit 3: Fail allowed. Bit 4: Retry allowed. Bit 7: 0 (a disk error).
AL	Drive: 0 = A:, 1 = B:.
DI	Low byte: 00h write-protected, 02h not ready, 04h data error, 07h unknown media, 08h sector not found, 0Ch general failure.
BP:SI	0000:0000 (no device header).

The handler returns with IRET and AL = 1 to retry, 2 to abort (end the program), or 3 (or 0) to fail the function, which then returns error 83. The handler runs on COS's own stack and must not call INT 21h.

System Limits

Drives	2 (A: and B:)
Files open at once	20 in all, 20 per program

Command line	127 characters
.COM program	65,280 bytes
Memory for programs	597,536 bytes in the largest block
Files per diskette	224 (1.44 MB), 112 (720 KB)
Dates	1 January 1980 to 31 December 2099

Compatibility

COS provides the version 3.30 services listed in Chapter 7, with the same numbers, registers and results. Programs that use only those services, through file handles, run on COS unchanged. COS Version 1.00 does not provide:

- File control block (FCB) file functions, and absolute disk access (INT 25h and 26h).
- Redirection (> and <), pipes (|), and the batch commands IF, GOTO, FOR, SHIFT and CALL, and parameters (%1) in batch files.
- Installable device drivers, CONFIG.SYS, and the printer and serial devices.
- Commands such as FORMAT, DISKCOPY, CHKDSK and SYS.
- Loading high: an .EXE file that asks for no memory above it is loaded low.
- A search path (PATH): programs are looked for in the current directory, and then in the root directory.
- A check that a diskette changed while a file was open on it ("Invalid disk change").

Programs that ask for a missing function are stopped with a message rather than given a wrong result.

9

EDIT, the Text Editor

CONTENTS

Introduction 9-2
EDIT 9-2
The Screen 9-2
Moving the Cursor 9-3
Typing and Changing Text 9-4
Moving Lines 9-4
Finding Text 9-4
Saving and Leaving 9-5
Text Files 9-5
EDIT Messages 9-5

Introduction

EDIT is the COS text editor. With it you write and change *text files*: programs in assembly language, letters, notes, lists. The file fills the screen; you move the cursor to where you want to work and type. What you type goes into the text at the cursor, and the screen always shows the text as it is.

EDIT works on a copy of the file in memory. Nothing changes on the diskette until you *save* the text with **F2**, and the old version of the file is kept, with the extension .BAK, in case you change your mind.

EDIT

PURPOSE

Creates a text file, or changes one.

FORMAT

EDIT [d:][*path*]*filename*[*.ext*]

REMARKS

Give the whole name of the file, with its extension: EDIT does not add one. If the file does not exist, EDIT starts with an empty screen and the message "New file"; the file is created when you save it.

When you leave EDIT, the screen is cleared and the COS prompt comes back.

EXAMPLE

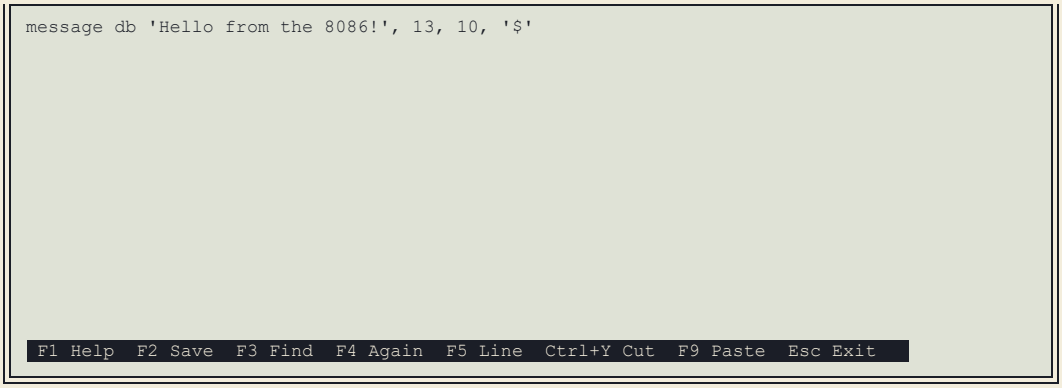
```
A:\>edit hello.asm
```

The Screen

This is what you see after EDIT HELLO.ASM, with the cursor moved to the fourth line:

```
 COS Edit  HELLO.ASM                          Line 4 of 9       Col 51    Insert
; hello.asm - the classic: print a string with COS service 09h
org 100h

    mov dx, message     ; DS:DX points to the text
    mov ah, 09h         ; "display a string ending in $"
    int 21h
    ret                 ; a .COM program may end with RET
```



The top line, in reverse video, shows the name of the file, followed by an asterisk (*) once you have changed the text and not yet saved it; where the cursor is, as a line number and a column number; and **Insert** or **Over**, the typing mode. The next 23 lines show the text. The bottom line lists the most used keys, and is where EDIT shows its messages and asks its questions.

Moving the Cursor

Key	Moves the cursor
↑ ↓	One line up or down. The cursor keeps its column when it can.
← →	One character left or right; past the start or the end of a line it goes to the line before or after.
Ctrl-← Ctrl-→	To the start of the word before or after.
Home End	To the start or the end of the line.
PgUp PgDn	One screen (22 lines) up or down.
Ctrl-Home Ctrl-End	To the first or the last line of the text.
F5	To a line by its number. EDIT asks "Go to line:"; type the number and press Enter. This is the quickest way to the line of an error reported by ASM.

The screen follows the cursor: when it goes past the top or the bottom, the text moves up or down; when it goes past column 80, the text moves sideways, so that long lines can be seen and changed.

Typing and Changing Text

Key	What it does
Characters	Go into the text at the cursor. In Insert mode the rest of the line moves right; in Over mode they replace the characters under the cursor.
Ins	Changes between Insert and Over. The cursor is a line in Insert mode and a block in Over mode.
Enter	Ends the line: what is after the cursor goes to a new line below. The new line starts with as many spaces as the line above starts with, so indented programs stay indented.
Backspace	Erases the character before the cursor. At the start of a line, joins the line to the one above.
Del	Erases the character at the cursor. At the end of a line, joins the next line to it.
Tab	Moves to the next column that is a multiple of 4, putting spaces in.

Moving Lines

Ctrl-**Y** cuts the line the cursor is on: the line leaves the text and is kept. Press it several times in a row and the lines pile up, in order. **F9** puts copies of the cut lines back into the text, before the line the cursor is on. The cut lines are kept until you cut again, so F9 can put them in several places.

To move a group of lines: put the cursor on the first, press Ctrl-Y once for each line, move the cursor to the line that must come after them, and press F9. To copy lines, cut them, press F9 where they were, then F9 again in the new place.

Finding Text

F3 asks "Find:" on the bottom line. Type the text you are looking for and press Enter (or Esc to give up). EDIT moves the cursor to the next place after the cursor where that text appears; capital and small letters are the same for the search. When it reaches the end of the text, it goes on from the top. **F4** finds the same text again. If the text is not there, EDIT says "Not found".

```
Find: message
```

Saving and Leaving

F2 saves the text. EDIT writes it to a new file, NAME.$$$; only when that has been written completely does the old file become NAME.BAK and the new one take the name of the file. If the diskette is full or write-protected, the old file is not touched, and EDIT says "Cannot save the file". The bottom line shows "Saved" when it is done, and you can go on working.

Esc (or F10) leaves EDIT. If the text has changed since it was last saved, EDIT asks first:

```
Save the changes (Y/N)?
```

Press Y to save the text and leave, N to leave without saving it (the file stays as it was on the diskette), or Esc to go back to the text.

F1 shows a page with all the keys; press any key to go back to the text.

Text Files

EDIT reads any file and treats it as lines of text. When it saves, every line ends with a carriage return and a line feed (0Dh 0Ah), the form COS and its programs expect. A tab character (09h) in a file becomes spaces, to the next column that is a multiple of 8. A Ctrl-Z character (1Ah) marks the end of the text: anything after it is not read.

A line can hold up to 1,000 characters; a longer line in a file is divided into lines of 1,000. The text can have up to 16,000 lines, and as many characters as fit in memory: about 450 KB on a computer with 640 KB.

EDIT Messages

Message	Meaning
New file	There is no file with that name. EDIT starts with an empty text; the file is created when you save.
Saved	The text is on the diskette.

Message	Meaning
Cannot save the file (the diskette is full or protected?)	The text could not be written. The old file is as it was. Make room on the diskette, or remove the write-protect tab, and press F2 again.
Not found	The text you asked F3 or F4 for is not in the file.
The line would be longer than 1000 characters	The character, or the joining of two lines, was not done.
Too many lines (16000 at most)	Enter or F9 would make the text longer than EDIT can hold.
Too many cut lines (500 at most)	Put the cut lines back with F9 before cutting more.
Lines longer than 1000 characters were split	The file had longer lines; each was divided. Saving the text saves them divided.
Out of memory: save your work	The text has filled the memory: the line you are on cannot grow and the cursor cannot leave it. Press F2 to save, which always works, then leave.
EDIT: the file is too large	The file does not fit in memory, or has more than 16,000 lines. EDIT ends without changing it.
EDIT: cannot open *name*	The name is not a valid file name, or the drive does not exist.
Usage: EDIT file	You did not give a file name.

10

ASM, the Assembler

CONTENTS

Introduction 10-2
Writing a Program 10-2
ASM 10-3
How ASM Works 10-4
Assembling COS 10-4
The Source File 10-5
ASM Messages 10-8

Introduction

A program for the 8086 is a sequence of instructions in machine code: bytes. Writing the bytes by hand is slow and full of mistakes, so programs are written instead in *assembly language*, where each instruction has a name (MOV, ADD, JMP) and each place in the program a label. ASM, the COS assembler, reads a program in assembly language from a text file, the *source file*, and writes the bytes, ready to run, to a .COM file.

COS itself, COMMAND.COM, EDIT and ASM are written in the language ASM accepts, and ASM can assemble all of them, itself included.

Writing a Program

A program is made in three steps: write it with EDIT, assemble it with ASM, and run it. The System Diskette has a small program to try, HELLO.ASM:

```
; hello.asm - the classic: print a string with COS service 09h
org 100h

    mov dx, message     ; DS:DX points to the text
    mov ah, 09h         ; "display a string ending in $"
    int 21h
    ret                 ; a .COM program may end with RET

message db 'Hello from the 8086!', 13, 10, '$'
```

Assemble it and run it:

```
A:\>asm hello
HELLO.COM: 31 bytes

A:\>hello
Hello from the 8086!

A:\>dir hello.*

 Volume in drive A is COS
 Directory of  A:\

HELLO    ASM      299   1-01-80  12:00a
HELLO    COM       31   1-01-80  12:00a
        2 File(s)   1412608 bytes free

A:\>
```

To make the program say something else, type **edit hello.asm**, change the text between the quotes, press F2 and Esc, and assemble and run it again. Chapter 6 explains how a COS program is organized, and Chapter 7 lists the services it can ask COS for.

ASM

PURPOSE

Assembles a program.

FORMAT

ASM [d:][*path*]*source*[*.ext*] [[d:][*path*]*output*[*.ext*]]

REMARKS

If the source has no extension, ASM adds .ASM. Without an output name, the bytes go to a file with the name of the source and the extension .COM, on the same drive. Give an output name to choose another, for example COS.SYS.

When the program has no errors, ASM writes the output file and displays its name and size. Otherwise it writes no file, displays each error with the line where it is, and tells how many errors it found; the first 100 are shown. An error is reported as:

> *NAME.ASM*(*line*): *what is wrong*

Go to that line with F5 in EDIT, correct it, and assemble again. ASM ends with return code 0 when it made the file, 1 when the program has errors, and 2 when it could not run (no source file, not enough memory, a full diskette).

EXAMPLE

```
A:\>asm bad
BAD.ASM(4): undefined symbol: 'mesage'
BAD.ASM(7): value 300 does not fit in a byte (-128..255)
2 errors

A:\>
```

Line 4 says **mesage** instead of **message**, and line 7 puts 300 in AL, which holds numbers up to 255.

How ASM Works

ASM reads the source file once, a piece at a time, and keeps it in memory in a compact form, with every name looked up. Then it goes through the program several times, or *passes*. In each pass it works out the address of every line; a jump to a label further on may need 2 bytes or 3, depending on how far the label turns out to be, so the passes go on until no address changes. ASM always chooses the shortest form of each instruction. The last pass writes the bytes.

Assembling HELLO.ASM takes about a second. Larger programs take longer: COS.SYS, about 5,600 lines, takes about a minute. A source file can be of any size that fits on a diskette; the program it makes can be up to 64 KB.

Assembling COS

The source files of COS are text files like any other. With them on a diskette in drive B:, COS can be assembled on COS. Drive B: is made the default drive, so that the files are read and written there, and ASM is run from drive A: by typing **a:asm**:

```
B:\>a:asm cos.asm cos.sys
COS.SYS: 12902 bytes

B:\>a:asm command.asm
COMMAND.COM: 7035 bytes

B:\>a:asm asm.asm
ASM.COM: 17945 bytes

B:\>dir

 Volume in drive B is SOURCES
 Directory of  B:\

COS      ASM   112041   1-01-80  12:00a
COMMAND  ASM    40775   1-01-80  12:00a
BOOT     ASM     4149   1-01-80  12:00a
ASM      ASM   125177   1-01-80  12:00a
EDIT     ASM    40762   1-01-80  12:00a
COS      SYS    12902   1-01-80  12:01a
COMMAND  COM     7035   1-01-80  12:01a
ASM      COM    17945   1-01-80  12:02a
        8 File(s)   1095680 bytes free
```

```
B:\>
```

The three take between two and three minutes in all. The files made are byte for byte the same as those on the System Diskette. A new System Diskette is then made by copying COS.SYS, COMMAND.COM, EDIT.COM and ASM.COM onto a diskette that has the COS boot record, for example a copy of the System Diskette.

The Source File

A source file is a text file. Each line holds at most one instruction or directive, in this order, each part optional:

```
label:    instruction   operands      ; comment
```

- A **label** names the address of the line, so that other lines can use it. It starts with a letter, _, ?, @ or a period, and goes on with letters, digits and _ ? @ $ # ~ .; capital and small letters are different (Start and start are two labels). The colon may be left out when an instruction follows.
- A label that starts with a period is **local**: it belongs to the last label before it without a period, so .loop after **print:** is really **print.loop**, and each routine can have its own .loop. A label that starts with two periods is not local.
- The **instruction** is an 8086 instruction or a directive. Instruction names, register names and directives may be written in capital or small letters.
- Operands are separated by commas.
- A semicolon starts a **comment**: the rest of the line is for the reader.

Numbers, Characters and Strings

Written	Meaning
200, 0d200, 200d	Decimal.
0C8h, 0xC8, $0C8	Hexadecimal. A number must start with a digit: write 0FFh, not FFh, which would be a label.
11001000b, 0b11001000	Binary. An underscore may separate the digits: 1100_1000b.
310o, 310q	Octal.

Written	Meaning
'A', "A"	A character: its code, 41h. Two characters, 'AB', make a word (41h, then 42h in memory).
'Hello, world'	A string, in DB and DW.

Expressions

Wherever a number goes, an expression may go. ASM computes it with 32 bits. The operators, from the first done to the last:

Operator	Meaning
( )	Grouping
- + ~ !	Negative, positive, NOT of each bit, logical NOT (1 if zero, else 0)
*** / % // %%**	Multiply; divide and remainder without sign; divide and remainder with sign
+ -	Add, subtract
<< >>	Shift left, right
&	AND
^	Exclusive OR
|	OR

Two symbols have a special meaning: **$** is the address of the start of the current line, and **$$** the address where the current section starts (for .text, the ORG). So **$ - message** is the number of bytes from the label message to here, and **times 510 - ($ - $$) db 0** fills a boot sector up to byte 510.

Directives

Directive	What it does
ORG *address*	The address of the first byte: 100h for a .COM program, 0 for COS.SYS, 7C00h for a boot sector. Once, before any code. Without it, the program starts at 0.
DB *values*	Bytes: numbers from -128 to 255, characters and strings, separated by commas. **db 'Hello', 13, 10, '$'**
DW *values*	Words (two bytes, the low one first), and strings padded to a whole number of words. **dw table, 1000**
RESB *n*, RESW *n*	*n* bytes or words of zeros.

Directive	What it does
name* EQU *value	Gives a name to a number. The name has no colon. **WIDTH equ 80**
TIMES *n line*	Repeats an instruction or DB/DW *n* times. **times 16 db 0**
SECTION .text **SECTION .data** **SECTION .bss**	What follows goes in that section; SEGMENT is the same. A program can go from one to another as often as it likes. In the output file the code of .text comes first, then .data, starting at the next address that is a multiple of 4; .bss comes after .data but is not in the file at all: its labels have addresses, and only RESB and RESW go in it. It is the place for large buffers, which then take no room on the diskette.
BITS 16, CPU 8086	Say that the program is for the 16-bit 8086. They are accepted and change nothing.

Operands

Operand	Example
Registers	AL CL DL BL AH CH DH BH (8 bits); AX CX DX BX SP BP SI DI (16 bits); ES CS SS DS (segments)
Immediate: a value	mov cx, 10 · mov dx, message · mov al, 'A'
Memory: an address in brackets, made of BX or BP, SI or DI, and a displacement, in any order	mov al, [bx] · mov ax, [count] · mov [bp+si+4], cx · mov al, [table+di]
A segment other than the usual one, inside the brackets	mov al, [es:di] · mov ax, [cs:saved]
BYTE or WORD, when the size cannot be told from a register	inc byte [count] · mov word [bx], 0
SHORT, NEAR, FAR on jumps	jmp short again · call far [vector] · jmp 0F000h:0FFF0h

Instructions

ASM accepts every 8086 instruction that the pc86 computer runs:

Moving data	MOV XCHG LEA LDS LES PUSH POP PUSHF POPF LAHF SAHF CBW CWD IN OUT
Arithmetic	ADD ADC SUB SBB CMP INC DEC NEG MUL IMUL DIV IDIV
Logic and shifts	AND OR XOR NOT TEST SHL SAL SHR SAR ROL ROR RCL RCR

Jumps	JMP CALL RET RETN RETF; JO JNO JB JC JNAE JAE JNB JNC JE JZ JNE JNZ JBE JNA JA JNBE JS JNS JP JPE JNP JPO JL JNGE JGE JNL JLE JNG JG JNLE; LOOP LOOPE LOOPZ LOOPNE LOOPNZ JCXZ
Strings	MOVSB MOVSW CMPSB CMPSW STOSB STOSW LODSB LODSW SCASB SCASW, with REP REPE REPZ REPNE REPNZ
Interrupts	INT INT3 IRET CLI STI
Flags and others	CLC STC CMC CLD STD NOP HLT

On the 8086 a shift or rotate moves by 1 or by CL (**mov cl, 4** / **shl ax, cl**); PUSH takes a register or memory, not a number; and a conditional jump reaches only 128 bytes back or 127 forward: to go further, jump on the opposite condition over a JMP. ASM says so when a program tries otherwise.

The decimal arithmetic instructions (AAA, AAD, AAM, AAS, DAA, DAS), XLAT, LOCK, WAIT and ESC are not supported by the pc86 processor, and ASM rejects them, as it rejects the instructions of later processors (PUSHA, ENTER, MOVZX...).

What ASM Does Not Have

ASM Version 1.00 has no macros, no INCLUDE and no conditional assembly: a program is one source file, assembled into one segment, like a .COM file. Only the .text, .data and .bss sections exist, without attributes (ALIGN=, VSTART=). DD and the other directives for larger data are not supported; use DB and DW.

ASM Messages

These are the messages you are most likely to see. Each error message is preceded by the name of the source file and the line, as shown above.

Message	Meaning
undefined symbol: '*name*'	There is no label or EQU with that name. Check the spelling, and capital and small letters.
'*name*' is already defined on line *n*	Two lines have the same label.
unknown instruction: *name*	The word is not an instruction or a directive. A label alone on a line needs its colon.

Message	Meaning
conditional jump out of range: target is *n* bytes away and the limit is -128..127	Jump on the opposite condition over a JMP to the label.
value *n* does not fit in a byte (-128..255)	Also: in a word (-32768..65535).
operation size not specified: use 'byte [...]' or 'word [...]'	Nothing tells ASM whether the memory operand is a byte or a word.
operand sizes do not match	For example MOV AX,BL: one operand is a word, the other a byte.
invalid combination of operands for *name*	The 8086 has no form of the instruction with those operands.
the 8086 can only address memory through BX, BP, SI and DI	Only these registers can be used in brackets; also: only BX or BP with SI or DI.
the 8086 can only shift by 1 or by CL	Load the count into CL first.
***name* does not exist on the 8086 (it is from a later CPU)**	Write it with 8086 instructions.
instruction not supported on this machine: *name*	The pc86 processor does not run it.
invalid number: *text*	A word that starts with a digit is not a number: a hexadecimal number needs its h.
unterminated string (missing ')	The quote that ends the string is missing.
only RESB and RESW go in .bss	.bss is not in the file, so it can hold no code or data, only room.
the program does not fit in a 64 KB segment	The program is too large for one segment.
***n* errors**	The program has errors; no file was written.
ASM: cannot open *name*	There is no source file with that name.
ASM: cannot write *name*	The output file could not be written: the diskette is full or write-protected.
ASM: not enough memory	ASM needs 256 KB of free memory.
Usage: ASM source[.ASM] [output]	You did not give a source file.

This page intentionally left blank.

A

Appendix A. Error Codes

CONTENTS

Function Error Codes A-2
Return Codes A-2

Function Error Codes

Returned in AX, with the carry flag set, by the functions of Chapter 7.

Code	Meaning
1 (01h)	Invalid function or subfunction
2 (02h)	File not found
3 (03h)	Path not found: a directory of the path does not exist
4 (04h)	Too many open files: no handle left
5 (05h)	Access denied: read-only file, full directory, existing name, or a device the computer does not have
6 (06h)	Invalid handle
7 (07h)	Memory control blocks destroyed
8 (08h)	Insufficient memory
9 (09h)	Invalid memory block address
11 (0Bh)	Invalid format (an .EXE file that cannot be loaded)
12 (0Ch)	Invalid access code (function 3Dh)
15 (0Fh)	Invalid drive
16 (10h)	Attempt to remove the current directory (function 3Ah)
17 (11h)	Not the same device (function 56h)
18 (12h)	No more files (function 4Fh)
21 (15h)	Drive not ready
80 (50h)	File exists (function 5Bh)
83 (53h)	Fail: the user answered F to a critical error

Return Codes

Function 4Dh returns in AH how the last program ended: 00h normally, 01h by Ctrl-C, 02h by Abort after a critical error, 03h resident. AL is the code the program gave to function 4Ch or 31h, and 0 otherwise. A program ended by COS for an unsupported function returns FFh.

B

Appendix B. The Keyboard

CONTENTS

Keys at the Prompt B-2
Control Characters B-2
Extended Codes B-2

Keys at the Prompt

Enter	Ends the line.
Backspace	Erases the last character.
Esc	Cancels the line.
Ctrl-**C**	Cancels the line or stops the command or program.

Function and cursor keys are ignored at the prompt; programs can read them. The keyboard is the United States layout.

Control Characters

Holding Ctrl while pressing a letter gives the control characters 01h (Ctrl-A) to 1Ah (Ctrl-Z). Some have a name:

Code	Key	Name
03h	Ctrl-C	Break (stops a program)
07h	Ctrl-G	Bell
08h	Ctrl-H, Backspace	Backspace
09h	Ctrl-I, Tab	Tab
0Ah	Ctrl-J	Line feed
0Dh	Ctrl-M, Enter	Carriage return
1Ah	Ctrl-Z	End of file
1Bh	Esc	Escape

Extended Codes

Keys with no character return 00h, and then a second code, from functions 01h, 07h and 08h:

Key	Code	Key	Code
F1 to F10	3Bh to 44h	Home	47h
Up arrow	48h	PgUp	49h
Left arrow	4Bh	Right arrow	4Dh
End	4Fh	Down arrow	50h
PgDn	51h	Ins	52h
Del	53h	Shift-F1 to F10	54h to 5Dh

Key	Code	Key	Code
Ctrl-F1 to F10	5Eh to 67h	Alt-F1 to F10	68h to 71h

This page intentionally left blank.

C

Appendix C. The Character Set

CONTENTS

Code Page 437 ..... C-2

Code Page 437

The display shows these 256 characters. Find a character's code by its row (the first hexadecimal digit) and column (the second): A is 41h. Codes 00h to 1Fh print as symbols when written to video memory or with BIOS functions that do not act on them; through COS, the control characters of Appendix B act on the cursor instead.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0		☺	☻	♥	♦	♣	♠	•	◘	○	◙	♂	♀	♪	♫	☼
1	►	◄	↕	‼	¶	§	▬	↨	↑	↓	→	←	∟	↔	▲	▼
2		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	⌂
8	Ç	ü	é	â	ä	à	å	ç	ê	ë	è	ï	î	ì	Ä	Å
9	É	æ	Æ	ô	ö	ò	û	ù	ÿ	Ö	Ü	¢	£	¥	₧	ƒ
A	á	í	ó	ú	ñ	Ñ	ª	º	¿	⌐	¬	½	¼	¡	«	»
B	░	▒	▓	│	┤	╡	╢	╖	╕	╣	║	╗	╝	╜	╛	┐
C	└	┴	┬	├	─	┼	╞	╟	╚	╔	╩	╦	╠	═	╬	╧
D	╨	╤	╥	╙	╘	╒	╓	╫	╪	┘	┌	█	▄	▌	▐	▀
E	α	ß	Γ	π	Σ	σ	µ	τ	Φ	Θ	Ω	δ	∞	φ	ε	∩
F	≡	±	≥	≤	⌠	⌡	÷	≈	°	∙	·	√	ⁿ	²	■	

D

Appendix D. BIOS Services and the Hardware

CONTENTS

BIOS Services D-2
Hardware D-2

BIOS Services

Programs may call the BIOS of the pc86 computer for what COS does not do. These services are available:

INT	Services
10h	Video: AH = 00h set mode (03h text, 13h 320×200 graphics in 256 colors), 01h cursor shape, 02h/03h set/get cursor position, 05h page, 06h/07h scroll a window, 08h read character, 09h/0Ah write character, 0Eh teletype output, 0Fh get mode, 10h palette, 12h information, 13h write string, 1Ah display combination.
11h	Equipment list.
12h	Memory size: 640 (KB).
13h	Diskette: AH = 00h reset, 01h status, 02h read, 03h write, 04h verify, 05h format a track, 08h drive parameters, 15h drive type, 16h change line, 17h/18h set the type for format.
16h	Keyboard: AH = 00h/10h read a key, 01h/11h check for a key, 02h/12h shift state, 03h typematic rate, 05h store a key.
19h	Start the computer from drive A: again.
1Ah	Timer: AH = 00h get, 01h set the tick count (18.2 ticks per second since midnight).
1Ch	User timer hook, called on every tick. A program may take it over; restore it before ending.

Hardware

Ports	Device
20h-21h	Interrupt controller (8259A). IRQ 0 timer, 1 keyboard, 6 diskette.
40h-43h	Timer (8253): channel 0 the tick, channel 2 the speaker.
60h, 64h	Keyboard controller (8042).
61h	Speaker gate and enable.
00h-0Fh, 80h-8Fh	DMA controller (8237) and page registers.
3C0h-3DAh	VGA video card.
3F2h-3F7h	Diskette adapter (765).
B800:0000h	Text mode video memory: 80 × 25, character then attribute. Mode 13h: A000:0000h, a byte per dot.

D

Quick Reference

Command	What it does
d:	Makes d: the default drive
CD [d:][*path*]	Changes or displays the current directory
CLS	Clears the display
COPY *source* [*target*]	Copies files; global characters allowed
DATE [*mm-dd-yy*]	Displays or sets the date
DEL *filespec*	Erases files (also ERASE)
ASM *source* [*output*]	Assembles a program (Chapter 10)
BASIC *program*	Runs a CBasic program (see the CBasic manual)
DIR [*filespec*] [/W]	Lists files
ECHO [*message*]	Displays a message
EDIT *filespec*	Writes or changes a text file (Chapter 9)
MD [d:]*path*	Makes a directory
PAUSE	Waits for a key
RD [d:]*path*	Removes an empty directory
REN *old new*	Renames files (also RENAME)
SET	Displays the environment
TIME [*hh:mm*]	Displays or sets the time
TYPE *filespec*	Displays a text file
VER	Displays the version
VOL [d:]	Displays the volume label
***name* [*parameters*]**	Runs NAME.COM or NAME.EXE

Key	At the prompt
Backspace	Erase the last character
Esc	Cancel the line
Ctrl-**C**	Stop the command or program
Abort, Retry, Fail?	A: end the program · R: try again · F: give up the operation

Key	In EDIT
F2 · **Esc**	Save · leave
F3 · **F4** · **F5**	Find · find again · go to a line

Key	In EDIT
Ctrl-Y · F9	Cut the line · put the cut lines back
Ins · F1	Insert or type over · all the keys

Program	INT 21h
Display a string	AH=09h, DS:DX = text ending in '$'
Read a key	AH=08h (AL = key)
Open, read, write, close	AH=3Dh, 3Fh, 40h, 3Eh (BX = handle, CF = error)
End	AX=4C00h